

# 1. Introducción a JavaScript

JavaScript (JS) es un lenguaje de programación que permite añadir **interactividad y dinamismo** a las páginas web. Mientras que el HTML define la estructura y el CSS el estilo, JavaScript se encarga de:

- Actualizar contenido dinámicamente.
- Controlar archivos multimedia.
- Animar imágenes y gráficos.
- Gestionar eventos (clics, envíos de formularios, etc.).

## Características básicas demostradas en el código:

Al observar un script básico de JS, identificamos cinco pilares:

1. **Variables y constantes:** Para almacenar datos y estados (ej. `const name`).
2. **Funciones:** Bloques de código reutilizables para ejecutar acciones (ej. `function actualizarNombre()`).
3. **Acceso al DOM:** Capacidad de leer y escribir en el HTML y CSS (ej. `document.querySelector`).
4. **Detectores de eventos:** Escuchar acciones del usuario (ej. `addEventListener("click", ...)`).
5. **Funciones anónimas:** Funciones sin nombre que se pasan como argumentos.

---

# 2. El ecosistema de las APIs

Las **API (Application Programming Interfaces)** son conjuntos de bloques de código ya preparados que nos permiten realizar tareas complejas sin escribir todo desde cero.

| Tipo de API   | Descripción                                           | Ejemplos                                                                              |
|---------------|-------------------------------------------------------|---------------------------------------------------------------------------------------|
| Del Navegador | Integradas directamente en el software del navegador. | <b>DOM:</b> Modificar HTML/CSS.<br><br><b>Geolocalización:</b> Ubicación del usuario. |

|                    |                                                         |                                                                                        |
|--------------------|---------------------------------------------------------|----------------------------------------------------------------------------------------|
|                    |                                                         | <b>Canvas/WebGL:</b> Gráficos 2D/3D.<br><br><b>Audio/Vídeo:</b> Control de multimedia. |
| <b>De Terceros</b> | Proporcionadas por otras empresas o servicios externos. | Google Maps, OpenStreetMap, API de Bluesky.                                            |

### 3. Seguridad y Orden de Ejecución

#### Seguridad (Sandbox)

Por seguridad, cada pestaña del navegador funciona en un **entorno de ejecución independiente**. Esto impide que el código de un sitio web malicioso acceda a la información de otra pestaña (como tu banco o correo).

#### Orden de ejecución

JavaScript es un lenguaje **interpretado** que se lee de arriba abajo.

- **Importante:** Si intentas acceder a un elemento HTML antes de que este se haya cargado, el script fallará (devolverá **null**).
- **Solución:** Usar el evento **DOMContentLoaded** para asegurar que el HTML esté listo antes de ejecutar el código.

### 4. Estrategias de Carga y Ubicación

Existen tres formas de incluir JavaScript, ordenadas de mejor a peor práctica:

#### A. JavaScript Externo (Recomendado)

Se usa un archivo **.js** separado. Es más limpio y fácil de mantener.

- **Uso:** `<script src="script.js" defer></script>`
- **Atributo defer:** Permite que el HTML se siga analizando mientras se descarga el script, ejecutándolo solo al final.
- **Atributo type="module":** Los módulos también se cargan automáticamente tras el HTML, pero pueden dar problemas de **CORS** si se abren directamente desde el sistema de archivos (**file:///**) sin un servidor local.

## **B. JavaScript Interno**

El código se escribe dentro de etiquetas `<script>` en el propio HTML. Útil para scripts muy pequeños o específicos.

## **C. Controladores en línea (Obsoleto / Mala práctica)**

Escribir JS directamente dentro de atributos HTML (ej. `<button onclick="...">`). **No se debe usar** porque ensucia el código y dificulta la depuración.

---

# **5. Sintaxis de Comentarios**

Para que tu código sea legible y para estudiar mejor:

- `//` Comentario de una sola línea.
- `/*` Comentario de varias líneas `*/`

# 1. El Proyecto: Adivinar el Número

Este ejercicio sirve para entender la **lógica de control** y la **interacción con el usuario**. Los pasos principales son:

1. **Generar** un número aleatorio (\$1-100\$).
  2. **Controlar** los turnos del jugador.
  3. **Validar** la entrada y dar feedback (¿es mayor? ¿es menor?).
  4. **Finalizar** (ganar o agotar turnos) y permitir el **reinicio**.
- 

## 2. Fundamentos de Sintaxis en JS

### Variables y Constantes

- **let**: Declara variables cuyo valor **puede cambiar** (ej. el número de intentos).
- **const**: Declara constantes cuyo valor es **fijo** (ej. una referencia a un botón del DOM).

### Cadenas de texto (Strings)

- **Simples/Dobles (' o ")**: Texto plano estándar.
- **Plantillas literales (Backticks `)**: Permiten insertar variables directamente usando `${variable}`. Muy útiles para mensajes dinámicos como: ``Hola, ${nombre}``.

### Comparaciones y el operador Estricto

- **=== (Igualdad estricta)**: Compara valor y tipo de dato. Es la mejor práctica en JS para evitar errores lógicos (ej. evitar que el número 0 sea igual a una cadena vacía `""`).
- 

## 3. Estructuras de Control

### Condicionales (if-else)

Siguen la misma lógica que C++ o Java. Permiten ramificar la ejecución según el estado del juego.

### Bucles (for)

Existen dos formas principales de recorrer elementos:

1. **For clásico**: `for (let i = 0; i < 5; i++)` (Ideal para contadores numéricos).
  2. **For...of**: `for (const item of array)` (Ideal para recorrer colecciones de elementos, como una lista de párrafos).
-

## 4. Funciones

A diferencia de C++, en JS:

- **No se declara el tipo de retorno** (ni `int`, ni `void`).
- Empiezan con la palabra clave `function`.
- El `return` es opcional.

---

## 5. Gestión de Eventos

Los eventos son el puente entre el usuario y el código.

1. **Event Listener:** El "oído" que escucha la acción (ej. un clic).
2. **Manejador (Handler):** La función que se ejecuta cuando ocurre la acción.

**Ejemplo clave:**

JavaScript

```
// elemento.addEventListener("tipoDeEvento", funcionAManejar);
intentoAdivinar.addEventListener("click", checkGuess);
```

---

## 6. Manipulación Dinámica del DOM

El juego muestra cómo JavaScript puede cambiar la página en tiempo real:

### Crear y eliminar elementos:

- `document.createElement("button")`: Crea un botón en memoria.
- `document.body.append(boton)`: Lo añade visualmente a la página.
- `elemento.parentNode.removeChild(elemento)`: Lo elimina (forma clásica).

### Modificar propiedades y estilos:

- `campoIntento.disabled = true`:: Deshabilita un input para que el usuario no escriba más.
- `ultimoResultado.style.backgroundColor = "white"`:: Cambia el estilo CSS directamente desde JS.
- `campoIntento.focus()`:: Pone el cursor automáticamente en el cuadro de texto para mejorar la experiencia.

# 1. Tipos de Datos: Primitivos vs. Objetos

JavaScript divide sus datos en dos grandes grupos:

## A. Tipos Primitivos (Inmutables)

Son valores básicos que no se pueden modificar (aunque la variable que los contiene sí).

- **string**: Texto ("Hola").
- **number**: Números (enteros y decimales).
- **boolean**: `true` o `false`.
- **bigint**: Enteros muy grandes.
- **null**: Valor nulo intencionado.
- **undefined**: Variable declarada pero sin valor.
- **symbol**: Identificadores únicos.

**Auto-boxing**: JavaScript permite usar métodos (como `.toString()`) en primitivos envolviéndolos automáticamente en objetos temporales.

## B. Objetos (Mutables)

Son colecciones de propiedades (clave: valor). Se pueden añadir o quitar propiedades en tiempo de ejecución.

---

# 2. Declaración de Variables: `let`, `const` y Ámbitos

JavaScript tiene **tipado dinámico**: una variable puede ser un número y luego un texto.

- **let**: Para variables cuyo valor cambiará. Su ámbito es el **bloque** (entre `{ }`).
  - **const**: Para valores fijos. **Ojo**: Si es un objeto o array, puedes cambiar su contenido, pero no asignar un objeto nuevo a esa variable.
  - **var**: (Obsoleto) No lo uses, tiene problemas de ámbito (scope) que causan errores.
- 

# 3. Estructuras de Datos: Arrays y Objetos

## Arrays (Listas ordenadas)

Son objetos especiales. El índice empieza en **0**.

- **Propiedad clave**: `.length` (número de elementos).
- **Métodos de búsqueda**: `includes()`, `indexOf()`, `lastIndexOf()`.
- **Añadir/Eliminar**:
  - Final: `push()` (añade), `pop()` (quita).
  - Inicio: `unshift()` (añade), `shift()` (quita).
- **splice(pos, cant)**: Elimina elementos desde una posición.
- **join("; ")**: Convierte el array en texto separado por lo que elijas.

## Operador de Propagación (...)

Sirve para "desparramar" elementos.

- **Copiar:** `const copia = [...original];`
- **Combinar:** `const total = [...a, ...b];`

## Objetos (Diccionarios)

Acceso mediante punto (`obj.prop`) o corchetes (`obj["prop"]`).

- **Métodos:** Son funciones guardadas dentro de un objeto.
  - **this:** Se refiere al objeto actual. **Es obligatorio usarlo** para acceder a otras propiedades del mismo objeto.
- 

## 4. Operadores Especiales

| Operador          | Nombre            | Función                                                                       |
|-------------------|-------------------|-------------------------------------------------------------------------------|
| ===               | Igualdad estricta | Compara valor y tipo (Recomendado).                                           |
| ==                | Igualdad débil    | Compara valor tras convertir tipos (Evitar).                                  |
| ??                | Coalescencia nula | Devuelve el 2º valor si el 1º es <code>null</code> o <code>undefined</code> . |
| condition ? a : b | Ternario          | <code>if-else</code> en una sola línea.                                       |
| typeof            | Tipo              | Devuelve un string con el tipo de dato.                                       |
| in                | Pertenencia       | Comprueba si una <b>propiedad</b> (o índice) existe en un objeto/array.       |

---

## 5. Funciones y su Evolución

1. **Básica:** `function nombre(p1) { ... }`
2. **Anónima / Expresión:** `const miFunc = function() { ... };`
3. **Flecha (Arrow):** `(p1) => { ... }`
  - Si solo hay un parámetro, sobran paréntesis: `p1 => { ... }`
  - Si solo hay un `return`, sobra la palabra y las llaves: `x => x * 2`

### Paso de parámetros:

- Primitivos: Por **valor** (el original no cambia).
  - Objetos: Por **compartición** (si cambias una propiedad dentro de la función, el objeto original **sí cambia**).
- 

## 6. Eventos y el Objeto event

La forma correcta de trabajar: `elemento.addEventListener("tipo", manejador)`.

- **event.target:** Es el elemento que disparó el evento (clic, tecla, etc.). Muy útil en funciones genéricas.
  - **event.preventDefault():** Evita que el navegador haga lo normal (ej. evita que un formulario se envíe y recargue la página para poder validarlo tú primero con JS).
- 

## 7. Sentencias de Iteración

- **for clásico:** Control total con índice i.
- **while:** Hasta que se cumpla una condición.
- **for..of:** El mejor para recorrer Arrays o Strings directamente sin índices.
- **Programación Funcional (Moderno):**
  - `map():` Crea un nuevo array transformando cada elemento.
  - `filter():` Crea un nuevo array solo con los elementos que cumplen una condición.



# 1. El Navegador como Entorno de Ejecución

Desde JavaScript, el navegador no es una sola pieza, sino que se divide en tres objetos globales con responsabilidades distintas:

- **Navigator:** La identidad del navegador (idioma, acceso a webcam, estado de la batería).
  - **Window:** La pestaña actual. Controla el tamaño de la ventana, alertas y almacenamiento local.
  - **Document:** La página HTML cargada. Es la puerta de entrada al **DOM**.
- 

# 2. El DOM (Document Object Model)

Es una representación en forma de **árbol** de tu HTML.

- Cada etiqueta es un **Nodo de Elemento**.
  - Cada trozo de texto es un **Nodo de Texto** (`#text`).
  - **Jerarquía:** Tienes nodos **padres**, **hijos**, **descendientes** y **hermanos** (mismo nivel).
- 

# 3. Selección de Elementos (Acceso)

Para modificar algo, primero hay que "atraparlo".

- **querySelector("selector"):** El más potente. Usa sintaxis CSS (`#id`, `.clase`, `tag`). Devuelve el **primero** que encuentre.
  - **querySelectorAll("selector"):** Devuelve un `NodeList` con **todos** los que coincidan.
  - *Métodos antiguos:* `getElementById()` o `getElementsByTagName()`. Se usan menos hoy en día, pero son rápidos.
- 

# 4. Modificación y Estructura

Una vez tienes el elemento (ej. `const caja = document.querySelector(".box")`), puedes hacer:

| Acción        | Comando                                 | Nota                     |
|---------------|-----------------------------------------|--------------------------|
| Cambiar Texto | <code>elem.textContent = "Hola";</code> | Cambia el texto visible. |

|                         |                                            |                                                                             |
|-------------------------|--------------------------------------------|-----------------------------------------------------------------------------|
| <b>Cambiar Atributo</b> | <code>elem.href = "url";</code>            | Funciona con <code>src</code> , <code>id</code> , <code>value</code> , etc. |
| <b>Crear Nodo</b>       | <code>document.createElement("p");</code>  | Crea el elemento en memoria (aún no se ve).                                 |
| <b>Añadir al DOM</b>    | <code>padre.appendChild(nuevo);</code>     | Lo mete dentro del padre al final.                                          |
| <b>Mover Nodo</b>       | <code>padre.appendChild(existente);</code> | Si ya existía, lo "arranca" de donde estaba y lo mueve.                     |
| <b>Borrar Nodo</b>      | <code>elem.remove();</code>                | El elemento se borra a sí mismo.                                            |

## 5. Manipulación de Estilos (CSS)

Hay dos estrategias principales:

### A. Estilos en línea (`.style`)

Modificar el atributo `style` del HTML directamente.

- **Regla de oro:** En JS se usa `camelCase`.
  - CSS: `background-color`  $\rightarrow$  JS: `elem.style.backgroundColor`.
  - CSS: `text-align`  $\rightarrow$  JS: `elem.style.textAlign`.

### B. Gestión de Clases (`.classList`) - Recomendado

Es mejor tener el diseño en el CSS y solo "encender" o "apagar" clases con JS.

- `elem.classList.add("activa");`
- `elem.classList.remove("oculta");`
- `elem.classList.toggle("oscuro");` (Si la tiene la quita, si no la tiene la pone).

## 6. Jerarquía de Interfaces (Teoría de Ingeniería)

Esto es muy de examen. Todo elemento en JS hereda de clases superiores:

1. **Node**: La base de todo. Tiene propiedades como `childNodes`, `parentNode` y `textContent`.
  2. **Element**: Añade capacidades específicas de etiquetas (como atributos).
  3. **HTMLElement**: La base de todas las etiquetas HTML (`HTMLButtonElement`, `HTMLImageElement`, etc.).
- 

## 7. Propiedades de Navegación (Atajos)

Si ya tienes un nodo, puedes moverte por el árbol sin buscar de nuevo:

- `n.parentNode`: Ir al padre.
- `n.firstElementChild`: Ir al primer hijo que sea una etiqueta (ignorando textos vacíos).
- `n.nextElementSibling`: Ir al hermano de al lado.

**Importante para el examen:** Diferencia entre `childNodes` (incluye nodos de texto/espacios) y `children` (solo incluye etiquetas HTML). Normalmente querrás usar `children`.

# 1. El Problema: Sincronía y Bloqueo

JavaScript es **single-threaded** (monohebra). Esto significa que solo puede hacer una cosa a la vez.

- **Ejecución síncrona:** Las funciones se ejecutan una tras otra. Si una función tarda mucho (ej. procesar millones de datos), el navegador deja de responder y la página se bloquea.
  - **La solución asíncrona:** Permite iniciar una operación larga y recuperar el control inmediatamente para que la página siga respondiendo mientras la operación termina "en segundo plano".
- 

# 2. Temporizadores e Intervalos

Son las formas más básicas de asincronía basadas en tiempo.

| Método      | Función                                              | Ejemplo                        |
|-------------|------------------------------------------------------|--------------------------------|
| setTimeout  | Ejecuta una función <b>una vez</b> tras un retraso.  | setTimeout(() => {...}, 1000); |
| setInterval | Ejecuta una función <b>periódicamente</b> cada X ms. | setInterval(miFunc, 500);      |

- **Cancelación:** Ambos devuelven un ID que permite detenerlos usando `clearTimeout(id)` o `clearInterval(id)`.
- 

# 3. Retrollamadas (Callbacks)

Una **callback** es una función que pasas como argumento a otra para que se ejecute después.

- **Problema (Callback Hell):** Si anidas muchas operaciones asíncronas (paso 1 depende del 2, el 2 del 3...), el código se vuelve una pirámide imposible de leer y mantener. Por esto surgieron las **Promesas**.
-

## 4. Promesas (Promises)

Una promesa es un objeto que representa el **estado futuro** de una operación asíncrona.

- **Estados de una promesa:**
  1. **Pending (Pendiente):** Estado inicial.
  2. **Fulfilled (Completada):** La operación tuvo éxito. Se activa el método `.then()`.
  3. **Rejected (Rechazada):** La operación falló. Se activa el método `.catch()`.

### Consumo de Promesas con `fetch()`

La API `fetch()` es la forma moderna de pedir datos a un servidor. Reemplaza al antiguo `XMLHttpRequest`.

JavaScript

```
fetch(url)

.then(response => {

  if (!response.ok) throw new Error("Error HTTP"); // Control de error manual

  return response.json(); // json() también devuelve una promesa

})

.then(data => console.log(data)) // Aquí tenemos los datos finales

.catch(error => console.error(error)) // Captura cualquier error de la cadena

.finally(() => console.log("Finalizado")); // Se ejecuta siempre
```

---

## 5. Async / Await

Es la forma más moderna y legible de escribir código asíncrono. Hace que el código asíncrono **parezca síncrono**.

- **async:** Se pone antes de la función para indicar que devolverá una promesa.
- **await:** Se usa **dentro** de una función `async` para "pausar" la ejecución hasta que la promesa se resuelva.

JavaScript

```
async function obtenerDatos() {  
  try {  
    const response = await fetch(url); // Espera a la respuesta  
    const data = await response.json(); // Espera a la conversión JSON  
    console.log(data);  
  } catch (error) {  
    console.error("Hubo un error", error);  
  }  
}
```

---

## 6. Combinación de Promesas

A veces necesitas lanzar varias peticiones a la vez:

- **Promise.all([p1, p2]):** Espera a que **todas** terminen con éxito. Si una falla, todo el conjunto falla.
- **Promise.any([p1, p2]):** Se completa en cuanto **una sola** de ellas tenga éxito.

# 1. La Cadena de Prototipos (El núcleo de JS)

En lenguajes como Java, la herencia es entre *clases*. En JavaScript, la herencia es entre **objetos**. Todo objeto en JS tiene un enlace interno (una referencia oculta llamada `__proto__`) que apunta a otro objeto, su **prototipo**.

- **¿Cómo funciona?** Si le pides a un objeto una propiedad o método y no lo tiene, JavaScript lo busca automáticamente en su prototipo. Si allí tampoco está, busca en el prototipo del prototipo, y así sucesivamente.
  - **Fin de la cadena:** La cadena termina cuando se llega al objeto base `Object.prototype`, cuyo prototipo es `null`.
  - **Delegación vs Herencia:** En lugar de copiar atributos (como hace la herencia clásica), JavaScript *delega* la tarea al objeto prototipo.
  - **Ocultación (Shadowing):** Si un objeto define un método con el mismo nombre que uno de su prototipo, el método del objeto "tapa" o anula al del prototipo.
- 

## 2. Formas de Crear y Heredar Objetos

Hay varias maneras de gestionar los prototipos.

### A. Constructores Funcionales (Forma clásica de JS)

Se crea una función normal, pero se invoca con la palabra `new`. Los métodos se asignan a la propiedad `.prototype` de la función para no copiar la misma función en cada objeto.

JavaScript

```
function Persona(nombre) {  
  
    this.nombre = nombre; // Propiedad propia del objeto  
  
}  
  
// El método se pone en el prototipo, se comparte con todos  
Persona.prototype.saludar = function() {  
  
    console.log(`Hola, soy ${this.nombre}`);  
  
};  
  
const fran = new Persona("Fran");
```

## B. Usando `Object.create()` (Forma ágil)

Permite crear un nuevo objeto y decirle directamente quién quieres que sea su prototipo, sin usar funciones constructoras.

JavaScript

```
const prototipoAnimal = {  
  respirar: function() { console.log("Respirando..."); }  
};  
  
// Crea un objeto vacío que tiene a 'prototipoAnimal' como su prototipo  
  
const miPerro = Object.create(prototipoAnimal);  
  
miPerro.respirar(); // Funciona por delegación
```

---

## 3. Clases en JavaScript (Sintaxis Moderna)

Para facilitar la vida a los programadores de Java/C++, JavaScript introdujo la sintaxis de `class`. *Es importante saber que, por debajo, esto sigue siendo una cadena de prototipos.*

### Sintaxis Básica y Constructores

- Usa `class NombreClase`.
- El método de inicialización se llama estrictamente `constructor()`.
- Es obligatorio usar `this` para referirse a las propiedades de la clase.

JavaScript

```
class Persona {  
  
  constructor(nombre) {  
  
    this.nombre = nombre; // Inicialización  
  
  }  
}
```



```
presentacion() {  
    console.log(`Hola, me llamo ${this.nombre}`);  
}  
}  
  
let p = new Persona("Federico"); // Creación de instancia
```

## Herencia Clásica (**extends** y **super**)

- Se usa la palabra **extends** para heredar de otra clase.
- En el constructor de la clase hija, **debes llamar a `super()`** antes de usar **this** para inicializar las propiedades del padre.

JavaScript

```
class Profesor extends Persona {  
    constructor(nombre, imparte) {  
        super(nombre); // Llama al constructor de Persona  
        this.imparte = imparte;  
    }  
    // Se pueden redefinir (sobreescribir) métodos  
}
```

---

## 4. Encapsulación y Elementos Estáticos

### Propiedades Privadas (**#**)

En JS tradicional todo era público. Ahora, si pones un **#** delante del nombre de una propiedad o método, se vuelve **privado**. Nadie desde fuera de la clase podrá leerlo o modificarlo.

JavaScript

```
class Estudiante {  
  
  #curso; // Propiedad privada  
  
  constructor(curso) {  
  
    this.#curso = curso;  
  
  }  
  
}
```

## Miembros Estáticos (**static**)

Los métodos o variables marcadas con **static** pertenecen a la clase misma, no a los objetos individuales (instancias). Se usan para herramientas o datos globales.

JavaScript

```
class Punto {  
  
  static nombreMostrado = "Punto";  
  
  static distancia(a, b) { ... }  
  
}  
  
const p1 = new Punto(5, 5);  
  
// p1.nombreMostrado daría undefined  
  
// Hay que llamarlo directamente a la clase:  
  
console.log(Punto.nombreMostrado); // Funciona
```

# 1. ¿Qué es JSON?

**JSON (JavaScript Object Notation)** es un formato de texto estándar utilizado para representar datos estructurados.

- **Origen:** Está basado en la sintaxis de los objetos de JavaScript, pero es **independiente del lenguaje** (Java, Python, C++, etc., pueden leerlo y escribirlo).
  - **Uso principal:** Transmisión de datos a través de la red (entre un Cliente/Navegador y un Servidor).
  - **Archivos:** Se guardan con la extensión **.json**.
  - **MIME type:** Cuando se envía por la red, se identifica como **application/json**.
- 

## 2. Estructura de JSON

Se representa siempre como una gran cadena de texto (**string**). Puede tener dos formas principales en su nivel más exterior:

### A. Como un Objeto { }

Colección de pares **clave/valor** separados por comas.

JSON

```
{  
  "nombre": "Juana",  
  "anyoNacimiento": 1516,  
  "titulos": ["Reina de Aragón", "Condesa de Barcelona"]  
}
```

### B. Como un Array [ ]

Lista de valores (que pueden ser objetos, otros arrays, o datos simples) separados por comas.

JSON

```
[  
  { "nombre": "Trastámara", "inicio": 1474 },  
  { "nombre": "Austria", "inicio": 1506 } ]
```

### 3. Acceso a los Valores (Desde JavaScript)

Una vez que esa cadena de texto JSON se ha convertido en un objeto real de JavaScript, interactúas con él exactamente igual que con cualquier otro objeto o array:

- **Operador punto (.)**: Para acceder a las propiedades (ej. `reina.nombre`).
  - **Corchetes ([ ])**: Para acceder a las posiciones de un array (ej. `reina.titulos[2]`).
- 

### 4. Restricciones de JSON (¡Muy importante para exámenes!)

*Todo JSON es un objeto JavaScript válido, pero NO todo objeto JavaScript es un JSON válido. JSON es mucho más estricto.*

**Lo que NO está permitido en JSON:**

1. **Tipos prohibidos**: No puedes usar `undefined`, `NaN`, ni `Infinity`.
  2. **Objetos complejos**: No se permiten funciones (`() => {}`), ni objetos de tipo `Date`, `Set`, o `Map`.
  3. **Comillas simples**: Todas las cadenas de texto **deben** usar comillas dobles `" "`. Las comillas simples `' '` darán error.
  4. **Claves sin comillas**: En JS puedes escribir `{ nombre: "Juan" }`. En JSON, la clave tiene que ir obligatoriamente entre comillas dobles: `{"nombre": "Juan"}`.
  5. **Comas finales**: No puedes dejar una coma "colgando" en el último elemento de un objeto o array. (ej. `[1, 2, 3, ]` es inválido).
  6. **Comentarios**: A diferencia de JS, en un archivo `.json` no puedes escribir `//` ni `/* */`.
- 

### 5. Conversión (Serialización y Deserialización)

Como JSON es solo texto, necesitas traducirlo a objetos reales de JS para usarlo, y viceversa. Hay dos formas principales de hacerlo:

#### A. La Interfaz global **JSON**

Tiene dos métodos estáticos fundamentales:

**JSON.parse(texto):** Convierte una cadena de texto JSON en un objeto de JavaScript.

JavaScript

```
const stringJSON = '{"nombre": "Felipe"}';
```

```
const objetoJS = JSON.parse(stringJSON); // Ahora es un objeto real
```

- 

**JSON.stringify(objeto):** Convierte un objeto de JavaScript en una cadena de texto JSON (ideal para enviarlo por la red o guardarlo).

JavaScript

```
const miObjeto = { nombre: "Carlos" };
```

```
const textoJSON = JSON.stringify(miObjeto); // Ahora es un string
```

- 

## B. El método **json()** de Fetch

Cuando haces una petición a un servidor usando **fetch()**, la respuesta llega como un flujo de datos (Stream). Para convertir automáticamente ese flujo de texto JSON en un objeto JavaScript, se usa el método **.json()** del objeto **Response** (recuerda que esto devuelve una promesa, como vimos en el tema de asincronía).

# 1. AJAX y la Descarga Dinámica

Históricamente, para actualizar un dato en una web, el navegador tenía que recargar la página entera (HTML, CSS, imágenes).

- **La técnica AJAX (Asynchronous JavaScript And XML):** Permite hacer peticiones HTTP en segundo plano. Solo se solicita el contenido dinámico necesario, actualizando la página usando el DOM sin necesidad de recargarla por completo.
  - Aunque sus siglas incluyen XML, **actualmente la respuesta suele venir en formato JSON** (también puede ser texto plano o HTML).
- 

## 2. La API `fetch()`

Es el estándar moderno para realizar estas peticiones. Funciona de manera asíncrona y **devuelve una Promesa**.

### Sintaxis Básica

JavaScript

```
fetch('https://api.ejemplo.com/datos')

.then(respuesta => {

  // 1. fetch() devuelve la promesa con un objeto Response

  if (!respuesta.ok) { // Comprueba si el código HTTP es 200-299

    throw new Error(`Error HTTP: ${respuesta.status}`);

  }

  // 2. Extraemos el cuerpo según su formato (esto devuelve otra promesa)

  return respuesta.json();

})

.then(datos => {

  // 3. Aquí manejamos los datos finales

  console.log(datos);

})

.catch(error => {
```

```
// 4. Captura errores de red (ej. sin internet) o errores lanzados con 'throw'

console.error("Fallo en la petición:", error);

});
```

## Métodos para procesar el cuerpo (Body) de la respuesta

Dependiendo de lo que pidas, debes extraer la respuesta con el método adecuado (todos devuelven una promesa):

- `respuesta.json()`: Para transformar texto JSON a un objeto JavaScript.
- `respuesta.text()`: Para texto plano o HTML.
- `respuesta.blob()`: Para archivos binarios (como imágenes, que luego puedes mostrar usando `URL.createObjectURL(blob)`).
- `respuesta.arrayBuffer()` y `respuesta.formData()`.

**Nota de Examen:** Un error 404 (Not Found) o 500 (Internal Server Error) **NO rechaza la promesa** inicial de `fetch`. La promesa solo se rechaza si hay un fallo de red (no hay internet, proxy bloqueado). Por eso es obligatorio comprobar siempre `if (!respuesta.ok)`.

---

## 3. Configuración Avanzada de Peticiones (Opciones)

`fetch()` acepta un segundo parámetro opcional: un objeto para configurar la petición.

### A. Cambiar el Método HTTP

Por defecto, `fetch` hace una petición `GET`.

JavaScript

```
fetch("https://api.com/usuarios", {

  method: "POST", // Puede ser POST, PUT, DELETE, etc.

  body: JSON.stringify({ nombre: "Hugo" }), // Los datos que enviamos

  headers: {

    "Content-Type": "application/json" // Le decimos al servidor el formato del cuerpo

  }
```

```
});
```

## B. Envío de Datos (**body** y Cabeceras)

- En peticiones **POST/PUT**: Los datos viajan en el **body**.
  - En peticiones **GET**: No se usa **body**. Los parámetros se añaden a la URL tras un signo de interrogación **?** (puedes usar `new URLSearchParams()`).
  - **Content-Type**: El navegador suele configurarlo automáticamente (por ejemplo, si envías **FormData** para subir un archivo). Sin embargo, si envías JSON como string, **debes** establecer manualmente `"Content-Type": "application/json"`.
- 

## 4. CORS (Cross-Origin Resource Sharing)

Esto es fundamental por seguridad. Un navegador, por defecto, bloquea las peticiones de un origen a otro origen distinto (distinto dominio, protocolo o puerto) a menos que el servidor de destino le dé permiso explícito.

- **Peticiones Simples**: El navegador hace la petición. Si la respuesta del servidor incluye la cabecera **Access-Control-Allow-Origin** adecuada (ej. **\*** o tu dominio específico), JavaScript recibe los datos. Si no, bloquea la respuesta.
  - **Peticiones No Simples (Preflight)**: El navegador envía primero una petición de verificación (método **OPTIONS**). Si el servidor confirma que entiende y acepta CORS, se envía la petición real.
  - **Modo **no-cors****: Limita las peticiones a GET/POST/HEAD y devuelve una **respuesta opaca**. JavaScript no podrá leer el cuerpo ni las cabeceras. Útil solo en escenarios muy concretos (como service workers).
- 

## 5. Credenciales (Cookies y Autenticación)

Por defecto, las peticiones de origen cruzado **NO** envían credenciales (cookies o cabeceras de autorización). Si necesitas enviarlas, usas la opción **credentials**:

- **omit**: Nunca se envían.
- **same-origin** (Por defecto): Se envían solo al mismo dominio.
- **include**: Se envían a otros dominios.

**Ojo con CORS + Credenciales**: Si usas `credentials: 'include'` en origen cruzado, el servidor debe responder con `Access-Control-Allow-Credentials: true` y **NO** puede usar



`Access-Control-Allow-Origin: *` (debe especificar el dominio exacto por seguridad).

---

## 6. Cancelar una petición

Si lanzas un `fetch` pero el usuario cambia de página o pulsas un botón de "Cancelar", puedes abortar la petición de red usando `AbortController`.

JavaScript

```
const controlador = new AbortController();
```

```
fetch("url", { signal: controlador.signal })
```

```
.catch(e => console.log("Petición abortada", e));
```

// Cuando quieras cancelarla:

```
controlador.abort();
```

# 1. ¿Por qué almacenar en el cliente?

Igual que el servidor guarda información en su base de datos, el navegador puede guardar datos localmente. Esto sirve para:

- **Personalización:** Recordar el idioma o el tema (claro/oscuro).
  - **Persistencia de sesión:** Mantener el inicio de sesión o el carrito de la compra activos.
  - **Rendimiento y Modo Offline:** Guardar datos localmente para que la web cargue más rápido o funcione sin conexión a internet.
- 

## 2. Cookies (El método tradicional)

Son pequeños fragmentos de información (máx. 4KB por dominio) que el servidor envía al navegador y que este **reenvía automáticamente al servidor en cada nueva petición**.

- **Usos:** Gestión de sesiones (IDs de sesión), personalización y seguimiento/tracking (comportamiento del usuario).
  - **Creación:** Generalmente se crean desde el servidor usando la cabecera HTTP `Set-Cookie`.
    - *Ejemplo:* `Set-Cookie: helado-preferido=fresa`.
  - **Expiración:** \* Si no se especifica (`Cookies de sesión`), se borran al cerrar el navegador.
    - Si se usa `Expires` o `Max-Age` (`Cookies permanentes`), duran hasta la fecha indicada.
    - Para borrar una cookie inmediatamente, se establece su `Max-Age` a `0` o su `Expires` a una fecha en el pasado.
  - **Seguridad (Muy importante para examen):**
    - `Secure`: Obliga a que la cookie solo viaje por HTTPS (conexiones seguras).
    - `HttpOnly`: Evita que JavaScript pueda leer la cookie (`document.cookie`), protegiendo la sesión contra ataques XSS (Cross-Site Scripting).
    - Modificar cookies desde JS (`document.cookie = "nombre=valor"`) solo debe hacerse para preferencias menores (ej. UI), **nunca** para tokens de autenticación.
- 

## 3. Web Storage API (`localStorage` y `sessionStorage`)

Es la alternativa moderna y sencilla a las cookies para guardar datos (pares clave-valor de texto) en el navegador **sin enviarlos al servidor**. Permiten guardar muchos más datos que las cookies.

- **localStorage**: Los datos persisten aunque se cierre el navegador y se apague el PC.
- **sessionStorage**: Los datos se borran en cuanto se cierra la pestaña o ventana actual.

#### Comandos básicos (Idénticos para ambos):

JavaScript

// Guardar/Actualizar

```
localStorage.setItem("usuario", "Hugo");
```

// Leer

```
let user = localStorage.getItem("usuario"); // Devuelve "Hugo"
```

// Borrar uno específico

```
localStorage.removeItem("usuario");
```

- **Nota de Seguridad**: La información está separada por dominio (una web no puede leer el **localStorage** de otra).

---

## 4. IndexedDB (Base de datos robusta)

Es una API compleja para crear una **base de datos orientada a objetos** dentro del navegador. Puede almacenar desde simples strings hasta archivos complejos (Blobs como imágenes, audios o vídeos).

### A. Conceptos Clave

- **Asincronía**: Todas las operaciones toman tiempo, por lo que devuelven peticiones que usan manejadores de eventos (**success**, **error**).
- **Almacenes de objetos (Object Stores)**: Es el equivalente a las "tablas" en SQL, pero guardan objetos de JavaScript.

- **Ruta de clave (`keyPath`):** Qué propiedad del objeto se usará como identificador único (ej. el `id`).
- **Transacciones:** Toda lectura/escritura debe hacerse dentro de una transacción (`readonly`, `readwrite` o `versionchange`).

## B. El evento vital: `upgradeneeded`

Este evento se dispara en dos ocasiones críticas:

1. Cuando se abre la base de datos por **primera vez**.
  2. Cuando cambias el **número de versión** al abrirla (ej. pasas de versión 1 a 2 porque necesitas añadir un nuevo almacén de objetos).
- *Aquí es el único lugar donde puedes crear la estructura (esquema) de la base de datos.*

## C. Flujo de Trabajo Básico (Para el examen práctico)

1. **Abrir BD:** `const petition = window.indexedDB.open("miBD", 1);`
2. **Crear Estructura:** En el manejador `upgradeneeded`, crear el almacén con `bd.createObjectStore("notas", {keyPath: "id"}).`
3. **Guardar Dato:** Abrir transacción `readwrite`, acceder al almacén y usar `.add(objeto)` o `.put(objeto)`.
4. **Leer Datos:** Abrir transacción, y usar `.get(clave)` o usar un **Cursor** (`openCursor()`) para recorrer todos los registros uno a uno.

# 1. ¿Qué son los Módulos y por qué usarlos?

Un módulo es simplemente un archivo JavaScript que exporta partes de su código (funciones, clases, variables) para que otros archivos puedan usarlas.

- **Beneficios:** Organización del código, reutilización y evitar conflictos de nombres (cada módulo tiene su propio "espacio" o ámbito).

## 2. Sintaxis de Exportación

Existen dos formas principales de exportar recursos:

### A. Exportaciones con Nombre (Named Exports)

Puedes exportar varios recursos en un mismo archivo. Obliga a importar usando exactamente el mismo nombre (salvo que uses `as`).

JavaScript

// utilidades.js

```
export const pi = 3.1416; // Exportar al declarar
```

```
function sumar(a, b) { return a + b; }
```

```
export { sumar }; // Exportar al final
```

### B. Exportación por Defecto (Default Export)

Solo puede haber **una** exportación por defecto por archivo. Es útil cuando un módulo hace una sola cosa principal (ej. una clase).

JavaScript

// calculadora.js

```
export default function multiplicar(a, b) { return a * b; }
```

```
// O al final: export default multiplicar;
```

---

### 3. Sintaxis de Importación

Dependiendo de cómo se exportó, se importa de una forma u otra:

#### A. Importar recursos con nombre (Llevar llaves `{}`)

JavaScript

```
import { pi, sumar } from './utilidades.js';
```

#### B. Importar el recurso por defecto (Sin llaves)

Puedes llamarlo como quieras al importarlo, no tiene que coincidir con el nombre original.

JavaScript

```
import cualquierNombre from './calculadora.js';
```

#### C. Solucionar conflictos de nombres (**as** o **\***)

Si dos módulos exportan una función llamada `dibujar`:

- **Renombrar al importar (**as**):** `import { dibujar as dibujarCirculo } from './circulo.js';`
- **Importar todo como un Objeto (**\***):** `import * as Circulo from './circulo.js';` (luego usas `Circulo.dibujar()`).

---

### 4. Características Únicas de los Módulos (Preguntas trampa de examen)

Si incluyes un módulo en el HTML, **debes** poner `type="module"`: `<script type="module" src="main.js"></script>`

**Diferencias con scripts normales:**

1. **Carga Diferida Automática:** Se comportan como si tuvieran `defer` (esperan a que el HTML cargue).
2. **Modo Estricto:** Siempre usan el "Strict Mode" de JS internamente.
3. **Ejecución Única:** Si importas un módulo 5 veces, su código se ejecuta solo 1 vez.
4. **CORS:** Necesitas usar un servidor local (Live Server) para probarlos; si abres el HTML haciendo doble clic (`file:///`), fallarán.

5. **Ámbito Privado:** Sus variables no se ensucian ni aparecen en la consola global del navegador.
- 

## 5. Conceptos Avanzados

**Agregación:** Un módulo "puente" que importa de varios sitios y vuelve a exportar todo junto para que el archivo final solo haga un `import`.

JavaScript

// figura.js (El agrupador)

```
export { Cuadrado } from './cuadrado.js';
```

- 

**Importación Dinámica:** Usa `import()` como si fuera una función. Es útil para no cargar código pesado hasta que el usuario hace click en un botón.

JavaScript

```
import('./modulo.js').then(Modulo => { ... });
```

- 

- **`await` en nivel superior (Top-level await):** Puedes usar `await` fuera de funciones `async` solo si estás dentro de un módulo. Esto pausa la ejecución del módulo que importa hasta que se resuelva la promesa, pero no bloquea el resto de la aplicación.

# Guia

| <u>Elemento</u>                                                                           | <u>¿Qué hace?</u>                                                                   | <u>Ejemplo</u>                                                                                     |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>let</code>                                                                          | Declara variables no globales.                                                      | <code>let dato</code>                                                                              |
| <code>const</code>                                                                        | Declara variables con distintos datos en su interior                                | <code>const libro = {<br/>  título: "El Quijote",<br/>  autor: "Cervantes",<br/>  ...<br/>}</code> |
| <code>console.log</code>                                                                  | Muestra por consola                                                                 | <code>console.log(dato)</code>                                                                     |
| <code>typeof</code>                                                                       | Muestra el tipo de la variable                                                      | <code>typeof dato</code>                                                                           |
| <code>??</code>                                                                           | SOLO actúa cuando se usa con "null" o "undefined"                                   | <code>let a = null / undefined<br/>console.log(a ?? "valor")<br/>(Muestra lo de la derecha)</code> |
| <code>  </code>                                                                           | Devuelve el valor de la derecha si es una variable: (false, 0, "", undefined, null) | <code>let a = 0<br/>console.log(a    "valor")<br/>(Muestra lo de la derecha)</code>                |
| <code>==</code>                                                                           | Compara valor, convierte los tipos                                                  | <code>let a = 5<br/>let b = "5"<br/>a == b → true</code>                                           |
| <code>===</code>                                                                          | Compara valor y tipo                                                                | <code>let a = 5<br/>let b = "5"<br/>a === b → false</code>                                         |
| <code>!=</code>                                                                           | Compara valor, convierte los tipos                                                  | <code>let a = 5<br/>let b = "5"<br/>a != b → false</code>                                          |
| <code>!==</code>                                                                          | Compara valor y tipo                                                                | <code>let a = 5<br/>let b = "5"<br/>a !== b → true</code>                                          |
| <code>const xxx = (x) =&gt; {}<br/>function x() {}<br/>const x =<br/>function() {}</code> | Se crean funciones                                                                  |                                                                                                    |
| <code>in</code>                                                                           | Comprueba si se encuentra dentro de la variable                                     | <code>const lista = [1,2,3,4,5]<br/>console.log(0 in lista)<br/>(devuelve false)</code>            |



|                                                  |                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cadenas                                          | Múltiples métodos                                                                                                            | <ul style="list-style-type: none"> <li>- <b>Length</b>: tamaño</li> <li>- <b>[num]</b>: posición</li> <li>- <b>includes()</b>: incluye</li> <li>- <b>startsWith()</b>: empieza por</li> <li>- <b>endsWith()</b>: termina por</li> <li>- <b>indexOf()</b>: posición de la letra</li> <li>- <b>slice()</b>: subcadena</li> <li>- <b>toLowerCase()</b>: todo a minúsculas</li> <li>- <b>toUpperCase()</b>: todo a mayúsculas</li> <li>- <b>replaceAll()</b>: reemplazar todas por otra cosa</li> <li>- <b>split()</b>: dividir por un elemento</li> <li>- <b>join()</b>: unir</li> </ul>                  |
| Arrays                                           | Múltiples métodos                                                                                                            | <ul style="list-style-type: none"> <li>- <b>Push</b>: añade al final</li> <li>- <b>Shift</b>: muestra el primero</li> <li>- <b>Unshift</b>: añade al principio</li> <li>- <b>Splice</b>: Elimina elemento</li> <li>- <b>includes()</b>: mira si lo incluye</li> <li>- <b>indexOf()</b>: posición primera</li> <li>- <b>lastIndexOf()</b>: posición última</li> <li>- <b>filter()</b>: se filtran por algo</li> <li>- <b>concat()</b>: juntamos dos arrays</li> <li>- <b>toString()</b>: pasamos a strings</li> <li>- <b>join()</b>: unimos por un carácter</li> <li>- <b>length</b>: tamaño</li> </ul> |
| <code>document.getElementById("id")</code>       | Permite asignar una variable a un id creado en el html                                                                       | <code>const boton = document.getElementById("boton")</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <code>addEventListener("evento", funcion)</code> | <b>Elemento</b> : objeto que va a escuchar<br><b>Evento</b> : acción que queremos detectar (click, keyup, submit, mouseover) | <code>boton.addEventListener("click", () =&gt; { console.log("¡El botón ha sido pulsado!"); document.body.style.backgroundColor = "lightblue"; });</code>                                                                                                                                                                                                                                                                                                                                                                                                                                              |

|                      |                                                               |                             |
|----------------------|---------------------------------------------------------------|-----------------------------|
|                      | <b>Función:</b> código que se ejecuta cuando ocurre el evento |                             |
| <code>alert()</code> | Muestra una alerta en pantalla                                | <code>alert("Error")</code> |

## 1. Interacción con el Usuario y Consola

Sirven para mostrar información o pedir datos rápidamente.

| Comando                    | Qué hace                                      | Ejemplo                                               |
|----------------------------|-----------------------------------------------|-------------------------------------------------------|
| <code>console.log()</code> | Imprime mensajes en la consola del navegador. | <code>console.log("Carga completa");</code>           |
| <code>alert()</code>       | Muestra una ventana emergente con un mensaje. | <code>alert("¡Has ganado!");</code>                   |
| <code>prompt()</code>      | Pide al usuario que introduzca un texto.      | <code>let nombre = prompt("¿Cómo te llamas?");</code> |

## 2. Selección de Elementos (DOM)

Sirven para que JavaScript "encuentre" los elementos de tu HTML.

| Comando                                  | Qué hace                                                                  | Ejemplo                                                            |
|------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>document.querySelector()</code>    | Selecciona el <b>primer</b> elemento que coincida (usa sintaxis CSS).     | <code>const btn = document.querySelector("#btn-enviar");</code>    |
| <code>document.querySelectorAll()</code> | Selecciona <b>todos</b> los elementos que coincidan (devuelve una lista). | <code>const parrafos = document.querySelectorAll(".texto");</code> |

|                                        |                                         |                                                                    |
|----------------------------------------|-----------------------------------------|--------------------------------------------------------------------|
| <code>document.getElementById()</code> | Selecciona un elemento por su ID único. | <code>const titulo = document.getElementById("main-title");</code> |
|----------------------------------------|-----------------------------------------|--------------------------------------------------------------------|

### 3. Manipulación de Elementos (DOM)

Sirven para cambiar el contenido, estilo o estructura de la página.

| Comando                               | Qué hace                                          | Ejemplo                                                      |
|---------------------------------------|---------------------------------------------------|--------------------------------------------------------------|
| <code>.textContent</code>             | Cambia o lee el texto dentro de un elemento.      | <code>titulo.textContent = "Nuevo Título";</code>            |
| <code>.value</code>                   | Obtiene o cambia el valor de un input/formulario. | <code>let email = campoEmail.value;</code>                   |
| <code>.style.propiedad</code>         | Cambia el CSS de un elemento directamente.        | <code>caja.style.backgroundColor = "red";</code>             |
| <code>document.createElement()</code> | Crea un nuevo elemento HTML en memoria.           | <code>const nuevoDiv = document.createElement("div");</code> |
| <code>.append()</code>                | Añade un elemento hijo al final de otro.          | <code>document.body.append(nuevoDiv);</code>                 |

### 4. Gestión de Eventos

Sirven para que la página "reaccione" a lo que hace el usuario.

| Comando                         | Qué hace                                                        | Ejemplo                                                |
|---------------------------------|-----------------------------------------------------------------|--------------------------------------------------------|
| <code>addEventListener()</code> | Escucha un evento (click, input, submit) y ejecuta una función. | <code>boton.addEventListener("click", saludar);</code> |
| <code>.disabled</code>          | Habilita o deshabilita botones o inputs.                        | <code>boton.disabled = true;</code>                    |
| <code>.focus()</code>           | Pone el cursor automáticamente en un campo de texto.            | <code>inputNombre.focus();</code>                      |

## 5. Lógica y Utilidades

Comandos internos para manejar datos.

| Comando                    | Qué hace                                          | Ejemplo                                         |
|----------------------------|---------------------------------------------------|-------------------------------------------------|
| <code>Math.random()</code> | Genera un número decimal aleatorio entre 0 y 1.   | <code>let azar = Math.random();</code>          |
| <code>Math.floor()</code>  | Redondea un número hacia abajo (hacia el entero). | <code>let entero = Math.floor(4.7); // 4</code> |
| <code>Number()</code>      | Convierte una cadena de texto a número.           | <code>let suma = Number("10") + 5; // 15</code> |
| <code>.push()</code>       | Añade un elemento al final de un Array (lista).   | <code>misNumeros.push(7);</code>                |